

MANUEL COMPLET DU PROJET

API Security Scanner

Comprendre chaque couche, chaque fichier, les flux, les tests et les limites
avant l'adaptation IASS / OT

Destinataire : Mathieu Morls

Périmètre : socle générique importé et stabilisé, avant toute adaptation OT.

Objectif : être capable d'expliquer l'architecture, le rôle des fichiers, le cheminement des données, les choix techniques, les commandes et les limites.

Document généré à partir de l'état local du dépôt au 19 août 2026.

Sommaire

Sommaire	2
1. Comment utiliser ce manuel	4
2. Résumé exécutif	4
3. Architecture et flux	4
3.1 Chemin réseau	4
3.2 Inscription et connexion	4
3.3 Cycle d'un scan	4
4. Modèle de données	5
5. Fichiers à la racine	6
6. Backend - lecture fichier par fichier	7
6.1 Entrées et configuration	7
6.2 Core	7
6.3 Modèles	8
6.4 Schémas	8
6.5 Repositories	8
6.6 Services	9
6.7 Routes	9
6.8 Scanners	9
7. Comprendre les quatre scanners	11
8. Frontend - lecture fichier par fichier	12
8.1 Configuration et entrée	12
8.2 Pages et routage	13
8.3 Composants	13
8.4 Hooks, services et stores	15
8.5 Bibliothèques, types et constantes	15
8.6 Styles globaux	16
8.7 Ressources publiques	16
9. Docker, Nginx et environnements	18
9.1 Différences dev / production	18

10. Documentation pédagogique learn/	19
11. Procédures de test	20
11.1 Démarrage standard	20
11.2 Vérifications opérationnelles	20
11.3 Parcours manuel sûr	20
11.4 Qualité frontend	20
11.5 Qualité backend	20
11.6 Arrêt et données	20
12. Sécurité et limites actuelles	21
13. Questions auxquelles savoir répondre	22
14. Glossaire rapide	23
15. État du dépôt et prochaine étape proposée	24
15.1 Ce que je proposerai après ton approbation	24
Annexe A - Répertoire généré et caches	25
Annexe B - Checklist de maîtrise	25

1. Comment utiliser ce manuel

Lis d’abord les chapitres 2 à 5 pour maîtriser la logique générale. Utilise ensuite les catalogues backend et frontend comme fiches de référence. Termine par les tests, limites, questions-réponses et la prochaine étape proposée.

Règle essentielle

Le scanner envoie de vraies requêtes vers une cible. Il ne doit être utilisé que sur un système que tu possèdes ou pour lequel tu disposes d’une autorisation explicite.

2. Résumé exécutif

Le dépôt contient une application web full-stack de test de sécurité d’API. Un utilisateur crée un compte, se connecte, renseigne une cible et sélectionne jusqu’à quatre familles de contrôles. FastAPI exécute les contrôles, PostgreSQL conserve les résultats et React les présente.

Brique	Technologie	Responsabilité
Interface	React 19 + TypeScript + Vite	Formulaires, navigation, historique et résultats
API	FastAPI + Pydantic	Validation, authentification, orchestration
Données	PostgreSQL + SQLAlchemy	Comptes, scans et résultats
Scanner	requests	Requêtes actives et preuves
Exécution	Docker Compose + Nginx	Isolation, réseau, proxy et production

3. Architecture et flux

3.1 Chemin réseau

```

Navigateur -> Nginx :80
/ -> React / Vite
/api/* -> FastAPI :8000 -> PostgreSQL
FastAPI -> HTTP vers l’API cible autorisée

```

En développement, le frontend peut aussi appeler directement `http://localhost:8000` selon `VITE_API_URL`. Dans le parcours unifié, `/api` est retiré par Nginx avant de transmettre la requête à FastAPI.

3.2 Inscription et connexion

- React valide le formulaire avec Zod.
- `POST /auth/register` crée un utilisateur dont le mot de passe est hashé avec `bcrypt`.
- `POST /auth/login` vérifie le hash et émet un JWT portant l’e-mail dans `sub`.
- Le frontend place JWT et identité d’affichage dans `localStorage`.
- Axios ajoute `Authorization: Bearer` aux appels suivants.

3.3 Cycle d’un scan

- `POST /scans/` passe par `get_current_user`.
- `ScanService` crée d’abord la ligne `Scan`.
- Chaque test sélectionné est associé à une classe `Scanner`.
- Les scanners s’exécutent séquentiellement dans la requête HTTP.
- Chaque résultat est enregistré avec statut, sévérité, preuves et recommandations.
- React Query invalide la liste et redirige vers `/scans/:id`.

4. Modèle de données

Table	Champs clés	Relations
users	id, email, hashed_password, is_active, dates	Un utilisateur possède plusieurs scans
scans	id, user_id, target_url, scan_date, dates	Un scan possède plusieurs résultats
test_results	scan_id, test_name, status, severity, details, JSON	Suppression avec le scan

L'accès aux scans est cloisonné par `user_id`. Le backend refuse la lecture ou la suppression d'un scan appartenant à un autre utilisateur. Les tables sont actuellement créées automatiquement au démarrage par SQLAlchemy.

5. Fichiers à la racine

.env.example

Modèle de configuration : identité de l'application, secret JWT, PostgreSQL, CORS, ports exposés et paramètres des scanners. Il doit être copié vers .env, qui reste privé.

.gitignore

Empêche Git d'ajouter les secrets, caches, dépendances, compilations, bases locales, journaux et réglages d'éditeur.

.dockerignore

Réduit et sécurise le contexte envoyé à Docker en excluant Git, .env, caches Python, node_modules et dist.

.pre-commit-config.yaml

Prévoit Ruff, mypy, pylint et des contrôles TOML/YAML avant commit. Il s'agit d'une barrière de qualité, pas de tests fonctionnels.

LICENSE

Licence GNU AGPL v3 : une distribution ou un service réseau modifié implique des obligations de partage du code selon la licence.

README.md

Page d'accueil du dépôt : finalité générique du scanner, pile technique, démarrage rapide et liens pédagogiques.

package.json

Fichier de commandes à la racine. Il ne porte pas l'application React : il sert surtout de façade pour démarrer, arrêter, reconstruire et consulter Docker.

justfile

Raccourcis équivalents organisés par groupes dev, production et nettoyage. Il charge le module backend/Justfile. L'outil just reste optionnel.

dev.compose.yml

Décrit les quatre services de développement : PostgreSQL, FastAPI avec rechargement, Vite avec HMR et Nginx proxy. Les sources sont montées en volumes.

compose.yml

Décrit la production : PostgreSQL privé, FastAPI sous Gunicorn et frontend React compilé puis servi par Nginx. Les images dev/prod ont des noms distincts.

6. Backend - lecture fichier par fichier

Le backend suit une séparation en couches : routes HTTP, services métier, repositories de données, modèles SQL, schémas de validation et scanners.

6.1 Entrées et configuration

backend/Justfile

Raccourcis backend basés sur uv : installation, qualité, formatage, nettoyage, recherche de TODO et mise à jour des dépendances.

backend/__init__.py

Marque backend comme paquet Python. Aucun comportement métier.

backend/config.py

Classe Settings Pydantic. Toutes les valeurs d'environnement, limites, délais, pagination, CORS et règles des scanners sont centralisées ici.

Classes : Settings. Fonctions : get_settings. Méthodes principales : Settings.cors_origins_list.

backend/factory.py

Fabrique FastAPI : crée les tables, configure la limitation entrante, CORS, les métadonnées OpenAPI, /, /health et les routeurs.

Fonctions : create_app, _register_routes.

backend/main.py

Point d'entrée minimal : charge les paramètres, appelle create_app(), expose app et permet un lancement direct Uvicorn.

backend/pyproject.toml

Métadonnées Python 3.13, dépendances d'exécution, outils de développement et règles Ruff, mypy, pylint, YAPF et Bandit.

6.2 Core

backend/core/__init__.py

Fichier d'initialisation de paquet : réexporte ou rend disponibles les éléments du dossier.

backend/core/database.py

Crée le moteur SQLAlchemy synchrone, la fabrique de sessions et la dépendance get_db qui ferme toujours la session.

Fonctions : get_db.

backend/core/dependencies.py

Lit le Bearer token, extrait le sujet e-mail, retrouve l'utilisateur et renvoie 401 si l'identité est invalide.

Fonctions : get_current_user.

backend/core/enums.py

Définit les valeurs strictes : statuts vulnerable/safe/error, sévérités et quatre types de test.

Classes : ScanStatus, Severity, TestType.

backend/core/security.py

Hash bcrypt, vérification des mots de passe, création des JWT signés et décodage avec contrôle de signature et expiration.

Fonctions : hash_password, verify_password, create_access_token, decode_token.

6.3 Modèles

backend/models/Base.py

Modèle SQLAlchemy abstrait : id, dates de création/mise à jour, conversion en dictionnaire et mise à jour générique.

Classes : BaseModel. Méthodes principales : BaseModel.__tablename__, BaseModel.to_dict, BaseModel.update, BaseModel.__repr__.

backend/models/Scan.py

Table scans : propriétaire, URL cible, date et relation vers les résultats. Suppression en cascade.

Classes : Scan. Méthodes principales : Scan.__repr__, Scan.has_vulnerabilities, Scan.vulnerability_count.

backend/models/TestResult.py

Table test_results : type, statut, sévérité, détails, preuve JSON et recommandations JSON.

Classes : TestResult. Méthodes principales : TestResult.__repr__, TestResult.is_vulnerable, TestResult.is_high_severity.

backend/models/User.py

Table users : e-mail unique indexé, mot de passe hashé et indicateur d'activité.

Classes : User. Méthodes principales : User.__repr__, User.is_authenticated.

backend/models/__init__.py

Fichier d'initialisation de paquet : réexporte ou rend disponibles les éléments du dossier.

6.4 Schémas

backend/schemas/__init__.py

Fichier d'initialisation de paquet : réexporte ou rend disponibles les éléments du dossier.

backend/schemas/scan_schemas.py

Contrats de création et lecture d'un scan. URL HTTP valide, au moins un test et max_requests entre 1 et 50.

Classes : ScanRequest, ScanResponse. Méthodes principales : ScanResponse.total_tests, ScanResponse.vulnerabilities_found.

backend/schemas/test_result_schemas.py

Contrats de création et restitution d'un résultat, avec preuves libres et recommandations.

Classes : TestResultCreate, TestResultResponse. Méthodes principales : TestResultResponse.is_vulnerable, TestResultResponse.is_high_severity.

backend/schemas/user_schemas.py

Contrats inscription, connexion, utilisateur public et réponse JWT. Valide la robustesse du mot de passe.

Classes : UserCreate, UserLogin, UserResponse, TokenResponse. Méthodes principales : UserCreate.validate_password_strength.

6.5 Repositories

backend/repositories/__init__.py

Fichier d'initialisation de paquet : réexporte ou rend disponibles les éléments du dossier.

backend/repositories/scan_repository.py

Persistance des scans, chargement avec résultats, pagination par utilisateur, scans récents, comptage et suppression.

Classes : ScanRepository. Méthodes principales : ScanRepository.create_scan, ScanRepository.get_by_id, ScanRepository.get_by_user, ScanRepository.get_recent, ScanRepository.delete, ScanRepository.count_by_user.

backend/repositories/test_result_repository.py

Création unitaire ou en lot, filtres par scan/statut et suppression des résultats.

Classes : `TestResultRepository`. Méthodes principales : `TestResultRepository.create_test_result`, `TestResultRepository.bulk_create`, `TestResultRepository.get_by_scan`, `TestResultRepository.get_by_status`, `TestResultRepository.get_vulnerabilities`, `TestResultRepository.delete_by_scan`.

backend/repositories/user_repository.py

Toutes les requêtes User : lecture, création, liste active, activation/désactivation et suppression.

Classes : `UserRepository`. Méthodes principales : `UserRepository.get_by_id`, `UserRepository.get_by_email`, `UserRepository.create_user`, `UserRepository.get_all_active`, `UserRepository.update_active_status`, `UserRepository.delete`.

6.6 Services

backend/services/__init__.py

Fichier d'initialisation de paquet : réexporte ou rend disponibles les éléments du dossier.

backend/services/auth_service.py

Logique d'inscription et de connexion : unicité e-mail, bcrypt, compte actif et émission du JWT.

Classes : `AuthService`. Méthodes principales : `AuthService.register_user`, `AuthService.login_user`, `AuthService.get_user_by_email`.

backend/services/scan_service.py

Orchestrateur : crée le scan, associe chaque type à sa classe, exécute séquentiellement, transforme les exceptions en résultat error et persiste.

Classes : `ScanService`. Méthodes principales : `ScanService.run_scan`, `ScanService.get_scan_by_id`, `ScanService.get_user_scans`, `ScanService.delete_scan`.

6.7 Routes

backend/routes/__init__.py

Fichier d'initialisation de paquet : réexporte ou rend disponibles les éléments du dossier.

backend/routes/auth.py

Expose POST /auth/register et POST /auth/login avec limitation par adresse cliente.

Fonctions : `register`, `login`.

backend/routes/scans.py

Expose création, liste, détail et suppression. Toutes ces routes exigent un utilisateur JWT valide et vérifient la propriété.

Fonctions : `create_scan`, `get_user_scans`, `get_scan`, `delete_scan`.

6.8 Scanners

backend/scanners/__init__.py

Fichier d'initialisation de paquet : réexporte ou rend disponibles les éléments du dossier.

backend/scanners/auth_scanner.py

Teste l'accès sans identité, les JWT alg=none, les signatures supprimées et plusieurs jetons invalides. Il juge principalement la racine de la cible.

Classes : `AuthScanner`. Méthodes principales : `AuthScanner.scan`, `AuthScanner._test_missing_authentication`, `AuthScanner._test_jwt_vulnerabilities`, `AuthScanner._test_none_algorithm`, `AuthScanner._test_signature_removal`, `AuthScanner._test_invalid_token_handling`, `AuthScanner._base64url_decode`, `AuthScanner._base64url_encode`, `AuthScanner._create_vulnerable_result`.

backend/scanners/base_scanner.py

Socle commun HTTP requests : session, Bearer facultatif, espacement, jitter, retries, Retry-After, mesures de temps et masquage des en-têtes sensibles.

Classes : BaseScanner. Méthodes principales : BaseScanner.__init__, BaseScanner._create_session, BaseScanner._wait_before_request, BaseScanner.make_request, BaseScanner.get_baseline_timing, BaseScanner.collect_evidence, BaseScanner._redact_sensitive_headers, BaseScanner.scan.

backend/scanners/idor_scanner.py

Extrait des id numériques/UUID de la réponse racine, tente d'autres identifiants et détecte les suites prévisibles.

Classes : IDORScanner. Méthodes principales : IDORScanner.scan, IDORScanner._test_id_enumeration, IDORScanner._extract_ids_from_response, IDORScanner._test_numeric_id_manipulation, IDORScanner._test_string_id_manipulation, IDORScanner._test_predictable_id_patterns, IDORScanner._create_vulnerable_result.

backend/scanners/payloads.py

Bibliothèque de chaînes SQL, jetons invalides, identifiants et variations de contournement. Certaines listes sont définies sans être utilisées actuellement.

Classes : SQLiPayloads, AuthPayloads, IDORPayloads, RateLimitBypassPayloads, XSSPayloads. Méthodes principales : SQLiPayloads.get_all_payloads, SQLiPayloads.get_error_signatures, AuthPayloads.get_all_headers, AuthPayloads.get_jwt_none_variants, IDORPayloads.get_numeric_tests, IDORPayloads.get_string_tests, RateLimitBypassPayloads.get_bypass_headers, RateLimitBypassPayloads.get_header_patterns, RateLimitBypassPayloads.get_endpoint_variations, XSSPayloads.get_all_payloads, XSSPayloads.get_basic_payloads.

backend/scanners/rate_limit_scanner.py

Cherche les en-têtes et réponses 429, puis tente des en-têtes IP falsifiés et variations de chemin.

Classes : RateLimitScanner. Méthodes principales : RateLimitScanner.scan, RateLimitScanner._detect_rate_limiting, RateLimitScanner._test_bypass_techniques, RateLimitScanner._test_ip_header_bypass, RateLimitScanner._test_endpoint_variation_bypass, RateLimitScanner._create_vulnerable_result.

backend/scanners/sqli_scanner.py

Envoie des paramètres id et recherche erreurs SQL, différences true/false et délais anormaux. Les conclusions sont heuristiques.

Classes : SQLiScanner. Méthodes principales : SQLiScanner.scan, SQLiScanner._test_error_based_sqli, SQLiScanner._test_boolean_based_sqli, SQLiScanner._test_time_based_sqli, SQLiScanner._create_vulnerable_result.

7. Comprendre les quatre scanners

Scanner	Ce qu'il fait réellement	Limite principale
Authentication	GET racine sans jeton, jetons invalides, JWT altéré	Un 200 sur une route publique peut devenir un faux positif
SQL Injection	Paramètre ?id= avec erreurs, couples vrai/faux et délais	Ne connaît ni les vrais paramètres ni les contrats métier
IDOR/BOLA	Extrait quelques id de la racine et tente des variantes	Sans endpoints objet explicites, couverture très faible
Rate Limiting	Recherche 429/en-têtes puis tente IP et variantes de chemin	Nombre de requêtes important et interprétation heuristique

Point technique important
max_requests règle principalement le rythme des requêtes. Le code ne l'utilise pas encore comme plafond global strict. Certaines boucles ont leur propre nombre d'itérations.

BaseScanner ajoute un jitter, espace les requêtes, suit request_count, respecte Retry-After et retente les erreurs de connexion ou serveur. Les preuves masquent certains en-têtes sensibles, mais pas tous les contenus potentiellement confidentiels.

8. Frontend - lecture fichier par fichier

Le frontend sépare présentation, appels réseau, état local, validation et types. Les fichiers CSS sont associés au composant ou à la page de même nom.

8.1 Configuration et entrée

frontend/.gitignore

Fichier de support frontend. Son rôle est déterminé par son emplacement et ses exports.

frontend/.npmrc

Réglages d'installation pnpm : pairs automatiques et builds de dépendances non bloquants.

frontend/.prettierrignore

Fichier de support frontend. Son rôle est déterminé par son emplacement et ses exports.

frontend/.prettierrc.json

Fichier de support frontend. Son rôle est déterminé par son emplacement et ses exports.

frontend/.stylelintrc.json

Fichier de support frontend. Son rôle est déterminé par son emplacement et ses exports.

frontend/biome.json

Formatage, qualité, sécurité et accessibilité frontend.

frontend/eslint.config.js

Seconde configuration de lint JS/React, disponible mais non utilisée par la commande lint principale.

Dépendances directes : @eslint/js, eslint-config-prettier, eslint-plugin-jsx-a11y, eslint-plugin-react, eslint-plugin-react-hooks, eslint-plugin-react-refresh, globals, typescript-eslint.

frontend/index.html

Document HTML racine, métadonnées, icônes et div #root utilisée par React.

frontend/package.json

Scripts frontend, dépendances React et outils de qualité. Épingle pnpm 10.34.5.

frontend/pnpm-lock.yaml

Résolution exacte et reproductible de toutes les dépendances transitives.

frontend/pnpm-workspace.yaml

Overrides de sécurité pnpm et autorisation du script de construction esbuild.

frontend/tsconfig.app.json

TypeScript strict pour src, DOM et JSX, sans génération de JS par tsc.

frontend/tsconfig.json

Point d'entrée TypeScript qui référence application et configuration Node.

frontend/tsconfig.node.json

TypeScript de vite.config.ts avec types Node.

frontend/vite.config.ts

Alias @, découpage des bundles, serveur 5173 et proxy /api vers FastAPI.

Dépendances directes : @vitejs/plugin-react, vite, vite-tsconfig-paths.

8.2 Pages et routage

frontend/src/App.tsx

Installe React Query, initialise la session depuis localStorage, le routeur, les notifications et les outils React Query.

Dépendances directes : @tanstack/react-query, @tanstack/react-query-devtools, react, react-router-dom, sonner, @/lib/queryClient, @/router, @/store/authStore.

frontend/src/main.tsx

Point d'entrée DOM : active StrictMode, monte App et charge les styles globaux.

Dépendances directes : react, react-dom/client, ./App.tsx.

frontend/src/pages/AuthPage.css

Feuille de style locale du composant ou de la page homonyme. Environ 2 blocs de règles : disposition, états, responsive et apparence.

frontend/src/pages/DashboardPage.css

Feuille de style locale du composant ou de la page homonyme. Environ 18 blocs de règles : disposition, états, responsive et apparence.

frontend/src/pages/DashboardPage.tsx

En-tête utilisateur, déconnexion, formulaire de nouveau scan et historique récent.

Exports : DashboardPage. Dépendances directes : react-router-dom, @/components/common/Button, @/components/scan/NewScanForm, @/components/scan/ScansList, @/store/authStore.

frontend/src/pages/LoginPage.tsx

Coquille de page qui affiche LoginForm.

Exports : LoginPage. Dépendances directes : @/components/auth/LoginForm.

frontend/src/pages/RegisterPage.tsx

Coquille de page qui affiche RegisterForm.

Exports : RegisterPage. Dépendances directes : @/components/auth/RegisterForm.

frontend/src/pages/ScanResultsPage.css

Feuille de style locale du composant ou de la page homonyme. Environ 21 blocs de règles : disposition, états, responsive et apparence.

frontend/src/pages/ScanResultsPage.tsx

Charge un scan par son id, gère chargement/erreur et affiche métadonnées, compteur et cartes de résultats.

Exports : ScanResultsPage. Dépendances directes : react-router-dom, @/components/scan/TestResultCard, @/hooks/useScan, @/lib/utlis.

frontend/src/router.tsx

Routes /login, /register, / et /scans/:id. Les deux dernières passent par ProtectedRoute.

Exports : router. Dépendances directes : react-router-dom, @/components/common/ProtectedRoute, @/pages/DashboardPage, @/pages/LoginPage, @/pages/RegisterPage, @/pages/ScanResultsPage.

8.3 Composants

frontend/src/components/auth/AuthForm.css

Feuille de style locale du composant ou de la page homonyme. Environ 10 blocs de règles : disposition, états, responsive et apparence.

frontend/src/components/auth/LoginForm.tsx

Formulaire contrôlé : validation locale, conservation temporaire, appel useLogin et affichage des erreurs.

Exports : LoginForm. Dépendances directes : axios, react, react-router-dom, @/components/common/Button, @/components/common/Input, @/hooks/useAuth, @/lib/validation, @/store/uiStore.

frontend/src/components/auth/RegisterForm.tsx

Inscription avec confirmation de mot de passe et mêmes validations que le backend.

Exports : RegisterForm. Dépendances directes : axios, react, react-router-dom, @/components/common/Button, @/components/common/Input, @/hooks/useAuth, @/lib/validation, @/store/uiStore.

frontend/src/components/common/Button.css

Feuille de style locale du composant ou de la page homonyme. Environ 13 blocs de règles : disposition, états, responsive et apparence.

frontend/src/components/common/Button.tsx

Bouton réutilisable avec variantes, état désactivé et indicateur de chargement.

Exports : Button. Dépendances directes : react.

frontend/src/components/common/Input.css

Feuille de style locale du composant ou de la page homonyme. Environ 10 blocs de règles : disposition, états, responsive et apparence.

frontend/src/components/common/Input.tsx

Champ réutilisable avec label, aide, erreur et attributs accessibles.

Exports : Input. Dépendances directes : react.

frontend/src/components/common/LoadingOverlay.css

Feuille de style locale du composant ou de la page homonyme. Environ 13 blocs de règles : disposition, états, responsive et apparence.

frontend/src/components/common/LoadingOverlay.tsx

Superposition pendant un scan, avec affichage des tests sélectionnés.

Exports : LoadingOverlay. Dépendances directes : @/config/constants.

frontend/src/components/common/ProtectedRoute.tsx

Attend l'initialisation du store puis redirige vers /login si aucune session locale.

Exports : ProtectedRoute. Dépendances directes : react-router-dom, @/store/authStore.

frontend/src/components/scan/NewScanForm.tsx

Saisie cible/token/tests/max_requests, validation Zod, persistance du brouillon et création du scan.

Exports : NewScanForm. Dépendances directes : react, @/components/common/Button, @/components/common/Input, @/components/common/LoadingOverlay, @/config/constants, @/hooks/useScan, @/lib/validation, @/store/uiStore.

frontend/src/components/scan/ScanForm.css

Feuille de style locale du composant ou de la page homonyme. Environ 11 blocs de règles : disposition, états, responsive et apparence.

frontend/src/components/scan/ScansList.css

Feuille de style locale du composant ou de la page homonyme. Environ 25 blocs de règles : disposition, états, responsive et apparence.

frontend/src/components/scan/ScansList.tsx

Tableau d'historique, compte les vulnérabilités et fournit un lien vers chaque résultat.

Exports : ScansList. Dépendances directes : react-icons/gi, react-router-dom, @/hooks/useScan, @/lib/utlis.

frontend/src/components/scan/TestIdCard.css

Feuille de style locale du composant ou de la page homonyme. Environ 20 blocs de règles : disposition, états, responsive et apparence.

frontend/src/components/scan/TestIdCard.tsx

Carte de détail avec badges, recommandations et preuve JSON extensible.

Exports : TestResultCard. Dépendances directes : @/config/constants, @/store/uiStore, @/types/scan.types.

8.4 Hooks, services et stores

frontend/src/hooks/useAuth.ts

Mutations React Query pour inscription/connexion, notifications, navigation et alimentation du store JWT.

Exports : useRegister, useLogin, useLogout. Dépendances directes : @tanstack/react-query, axios, react-router-dom, sonner, @/config/constants, @/services/authService, @/store/authStore, @/types/auth.types.

frontend/src/hooks/useScan.ts

Queries/mutations des scans, validation des réponses, invalidation du cache et navigation après création.

Exports : useGetScans, useGetScan, useCreateScan, useDeleteScan. Dépendances directes : @tanstack/react-query, axios, react-router-dom, sonner, @/config/constants, @/lib/errors, @/services/scanService, @/types/guards.

frontend/src/services/authService.ts

Appels POST vers register et login.

Exports : authQueryKeys, authMutations. Dépendances directes : @/config/constants, @/lib/api, @/types/auth.types.

frontend/src/services/scanService.ts

Appels GET/POST/DELETE des scans et clés de cache React Query.

Exports : scanQueryKeys, scanQueries, scanMutations. Dépendances directes : @/config/constants, @/lib/api, @/types/scan.types.

frontend/src/store/authStore.ts

Etat Zustand de session. JWT et pseudo-utilisateur sont conservés dans localStorage.

Exports : useAuthStore. Dépendances directes : zustand, zustand/middleware/immer, @/config/constants, @/types/auth.types.

frontend/src/store/uiStore.ts

Brouillons des formulaires et cartes ouvertes, persistés sept jours dans localStorage.

Exports : useUIStore. Dépendances directes : zustand, zustand/middleware, zustand/middleware/immer, @/config/constants.

8.5 Bibliothèques, types et constantes

frontend/src/config/constants.ts

URL de base API, routes, labels, couleurs, clés localStorage et messages d'erreur.

Exports : API_BASE_URL, AUTH_ENDPOINTS, AuthEndpoint, AUTH_ERROR_MESSAGES, AuthErrorMessage, AUTH_ERROR_CONTEXTS, AuthErrorContext, STORAGE_KEYS, APP_NAME, APP_VERSION, SCAN_TEST_TYPES, ScanTestType, TEST_TYPE_LABELS, SCAN_STATUS, ScanStatus, STATUS_COLORS, SEVERITY, Severity, SEVERITY_COLORS, SCAN_ENDPOINTS, SCAN_ERROR_MESSAGES, ScanErrorMessage, SCAN_ERROR_CONTEXTS, ScanErrorContext.

frontend/src/lib/api.ts

Instance Axios : baseURL, délai 180 s, ajout du Bearer token et purge de session sur 401.

Exports : api. Dépendances directes : axios, @/config/constants.

frontend/src/lib/errors.ts

Normalise les erreurs Axios et choisit un message compréhensible selon le contexte.

Exports : createApiErrorHandler. Dépendances directes : axios, sonner.

frontend/src/lib/queryClient.ts

Réglages globaux du cache et des nouvelles tentatives React Query.

Exports : queryClient. Dépendances directes : @tanstack/react-query.

frontend/src/lib/utils.ts

Fonctions d'affichage des dates et utilitaires de présentation.

Exports : formatDate, formatDateTime, formatRelativeTime.

frontend/src/lib/validation.ts

Schémas Zod des formulaires : e-mail, robustesse du mot de passe, URL, tests et limite 1-50.

Exports : loginSchema, registerSchema, scanSchema, LoginFormData, RegisterFormData, ScanFormData. Dépendances directes : zod.

frontend/src/types/auth.types.ts

Interfaces TypeScript des demandes/réponses d'authentification.

Exports : RegisterRequest, RegisterResponse, LoginRequest, LoginResponse, AuthUser.

frontend/src/types/guards.ts

Contrôles runtime : refuse une réponse serveur qui ne respecte pas les formes attendues.

Exports : isValidLoginResponse, isValidRegisterResponse, isValidCreateScanResponse, isValidGetScansResponse, isValidGetScanResponse. Dépendances directes : @/config/constants, ./auth.types, ./scan.types.

frontend/src/types/scan.types.ts

Interfaces Scan et TestResult alignées sur les réponses FastAPI.

Exports : TestResult, Scan, CreateScanRequest, CreateScanResponse, GetScansResponse, GetScanResponse. Dépendances directes : @/config/constants.

frontend/src/vite-env.d.ts

Déclarations de types fournies par Vite.

8.6 Styles globaux

frontend/src/styles/index.css

Normalisation et styles globaux de la page.

frontend/src/styles/variables.css

Variables CSS de couleurs, espacements, rayons, ombres et typographie.

8.7 Ressources publiques

frontend/public/android-chrome-192x192.png

Ressource graphique d'identité utilisée comme favicon ou icône mobile. Aucun code exécutable.

frontend/public/android-chrome-512x512.png

Ressource graphique d'identité utilisée comme favicon ou icône mobile. Aucun code exécutable.

frontend/public/apple-touch-icon.png

Ressource graphique d'identité utilisée comme favicon ou icône mobile. Aucun code exécutable.

frontend/public/favicon-16x16.png

Ressource graphique d'identité utilisée comme favicon ou icône mobile. Aucun code exécutable.

frontend/public/favicon-32x32.png

Ressource graphique d'identité utilisée comme favicon ou icône mobile. Aucun code exécutable.

frontend/public/favicon.ico

Ressource graphique d'identité utilisée comme favicon ou icône mobile. Aucun code exécutable.

frontend/public/site.webmanifest

Manifeste de l'application web : nom, icônes, couleurs et mode d'affichage.

9. Docker, Nginx et environnements

conf/docker/dev/fastapi.docker

Image Python 3.13, outils système, installation du projet puis Uvicorn avec rechargement.

conf/docker/dev/vite.docker

Image Node 20, pnpm 10 verrouillé, dépendances exactes puis serveur Vite accessible sur le réseau.

conf/docker/prod/fastapi.docker

Image backend de production, Gunicorn, utilisateur non-root et quatre workers Uvicorn.

conf/docker/prod/vite.docker

Construction multi-stage : TypeScript/Vite dans Node puis copie de dist dans une image Nginx légère.

conf/nginx/http.conf

Déclare les upstream backend:8000 et frontend:5173.

conf/nginx/dev.nginx

Proxy /api/ vers FastAPI et tout le reste vers Vite, avec WebSocket/HMR.

conf/nginx/prod.nginx

Sert les fichiers statiques, fallback React, cache des assets, en-têtes de sécurité, gzip et proxy /api/.

9.1 Différences dev / production

Sujet	Développement	Production
Frontend	Vite + HMR	Fichiers statiques Nginx
Backend	Uvicorn reload	Gunicorn, 4 workers
Base	Port 5432 exposé	Port non exposé
Sources	Volumes montés	Code intégré aux images
Secret	Valeur de secours non sûre	SECRET_KEY à fournir

10. Documentation pédagogique learn/

learn/00-OVERVIEW.md

Prérequis, objectif et prise en main du projet original.

À retenir : Document non exécuté et non encore adapté au futur périmètre OT.

learn/01-CONCEPTS.md

Concepts OWASP API, catégories de vulnérabilités et contexte sécurité.

À retenir : Document non exécuté et non encore adapté au futur périmètre OT.

learn/02-ARCHITECTURE.md

Architecture prévue, flux et séparation des composants.

À retenir : Document non exécuté et non encore adapté au futur périmètre OT.

learn/03-IMPLEMENTATION.md

Lecture guidée de l'implémentation générique.

À retenir : Document non exécuté et non encore adapté au futur périmètre OT.

learn/04-CHALLENGES.md

Idées d'exercices, extensions et défis.

À retenir : Document non exécuté et non encore adapté au futur périmètre OT.

11. Procédures de test

11.1 Démarrage standard

```
cp .env.example .env
docker compose -f dev.compose.yml up --build -d
```

Ouvrir ensuite <http://localhost>. La documentation FastAPI est disponible sur <http://localhost:8000/docs>.

11.2 Vérifications opérationnelles

```
docker compose -f dev.compose.yml ps
curl http://localhost/health
curl http://localhost/api/health
docker compose -f dev.compose.yml logs -f
```

11.3 Parcours manuel sûr

- Créer un compte de test.
- Se connecter et vérifier la redirection vers le tableau de bord.
- Utiliser <http://backend:8000> comme cible interne de démonstration.
- Sélectionner SQL Injection ou IDOR seulement pour un test court.
- Vérifier la redirection, les preuves et la présence du scan dans l'historique.
- Se déconnecter et vérifier que les pages protégées renvoient vers /login.

Interprétation

Un scan de <http://backend:8000> vérifie le pipeline, pas la pertinence du diagnostic. La racine de cette API est volontairement publique : le scanner Authentication la signalerait à tort.

11.4 Qualité frontend

```
cd frontend
npx --yes pnpm@10.34.5 install --frozen-lockfile
npx --yes pnpm@10.34.5 lint
npx --yes pnpm@10.34.5 typecheck
npx --yes pnpm@10.34.5 build
```

11.5 Qualité backend

```
cd backend
python3.13 -m venv .venv
.venv/bin/pip install -e '.[dev]'
.venv/bin/ruff check .
.venv/bin/mypy .
```

11.6 Arrêt et données

```
docker compose -f dev.compose.yml down
# Avec suppression irréversible des comptes et résultats :
docker compose -f dev.compose.yml down -v
```

Il n'existe encore ni suite pytest, ni tests de composants React, ni scénario end-to-end automatisé. Les validations actuelles couvrent surtout compilation, types, lint et smoke tests manuels.

12. Sécurité et limites actuelles

Limite	Conséquence
Scanner générique	Pas d'import OpenAPI, découverte de routes, verbes, corps JSON ou scénarios métier.
Faux positifs	Une réponse 200 sur une route publique peut être interprétée comme un défaut d'authentification.
Faux négatifs	Les tests ciblent surtout la racine et ?id=, sans connaître les paramètres réels.
Exécution synchrone	Une requête de scan longue immobilise la réponse HTTP et un worker.
Limite de requêtes	max_requests n'est pas un plafond global garanti.
SSRF et périmètre	Le backend accepte une URL arbitraire et ne possède pas encore de politique de cible autorisée.
Jeton navigateur	JWT stocké dans localStorage : simple, mais exposé en cas de XSS.
Migrations	Les tables sont créées automatiquement ; Alembic est déclaré mais pas intégré au démarrage.
Production	TLS, domaine, secret fort, sauvegardes et supervision restent à configurer.
Fonctions inutilisées	Socket.IO, certains payloads et plusieurs méthodes repository ne sont pas reliés à l'interface.

13. Questions auxquelles savoir répondre

Pourquoi FastAPI ?

Validation Pydantic, documentation OpenAPI automatique, injection de dépendances et structure claire pour une API Python.

Pourquoi PostgreSQL ?

Relations solides, transactions et JSON pour conserver des preuves variables.

Pourquoi trois couches route/service/repository ?

La route gère HTTP, le service porte le métier et le repository isole SQLAlchemy.

Comment un scan est-il protégé ?

HTTPBearer valide le JWT, retrouve l'utilisateur et ScanService vérifie user_id pour lecture/suppression.

Où le JWT est-il créé ?

backend/core/security.py, appelé par AuthService après vérification du mot de passe.

Où le JWT est-il envoyé ?

frontend/src/lib/api.ts l'ajoute automatiquement à Axios.

Pourquoi Nginx ?

Point d'entrée, routage /api, service statique en production, compression et en-têtes.

Les résultats sont-ils fiables ?

Ils sont heuristiques et pédagogiques. Ils nécessitent confirmation humaine et contexte métier.

Le scan est-il passif ?

Non. Les quatre scanners envoient activement des requêtes et des valeurs de test.

Que persiste le navigateur ?

JWT, identité d'affichage, brouillons de formulaire et état des cartes.

Que persiste Docker ?

Le volume PostgreSQL conserve comptes et scans même après docker compose down.

Pourquoi dev et prod ont-ils des images séparées ?

Pour empêcher qu'une construction de production remplace silencieusement l'image attendue en développement.

14. Glossaire rapide

Terme	Définition
API	Contrat permettant à des logiciels de communiquer.
Endpoint	Combinaison d'une URL et d'une méthode HTTP.
JWT	Jeton signé portant des informations d'identité et une expiration.
CORS	Règles déterminant quels sites peuvent appeler l'API depuis un navigateur.
ORM	Couche qui traduit objets Python et tables SQL.
Repository	Objet concentrant les accès à la base.
Schema	Contrat de données validé à l'entrée ou à la sortie.
HMR	Mise à jour instantanée du frontend en développement.
Reverse proxy	Serveur qui reçoit les appels et les transmet au bon service.
IDOR/BOLA	Accès à un objet sans contrôle suffisant de propriété.
SQLi	Injection d'une valeur qui modifie une requête SQL.
Rate limiting	Limitation du nombre d'appels autorisés sur une période.

15. État du dépôt et prochaine étape proposée

État actuel
Le socle fonctionne en développement, les images de production se construisent, mais tous les fichiers sont encore non suivis dans un dépôt sans premier commit. Aucun développement OT n'a commencé.

15.1 Ce que je proposerai après ton approbation

La prochaine étape ne devrait pas être de coder immédiatement tous les contrôles OT. Elle devrait être une phase de conception de l'adaptation IASS / OT, avec une matrice de correspondance entre le dossier, le socle et les écarts.

Sous-étape	Livrable attendu
1. Référentiel	Exigences du dossier classées : obligatoire, démonstration, preuve et hors périmètre.
2. Conservation	Fichiers du socle gardés, modifiés, remplacés ou supprimés, avec justification.
3. Architecture cible	Flux passifs/actifs, autorisations, inventaire, profils OT, résultats et audit.
4. Modèle de données	Nouvelles entités et relations avant toute migration.
5. Plan de réalisation	Lots ordonnés, critères d'acceptation et tests.

Décision attendue
Cette phase de conception commencera uniquement après ta lecture et ton approbation. Elle ne modifiera pas encore les scanners en profondeur.

Annexe A - Répertoire généré et caches

Ces dossiers peuvent être recréés et ne doivent pas être étudiés fichier par fichier :

Chemin	Origine	Peut être recréé par
frontend/node_modules/	Dépendances JavaScript	pnpm install
frontend/dist/	Compilation React	pnpm build
**/__pycache__/	Bytecode Python	interpréteur Python
backend/.mypy_cache/	Analyse de types	mypy
backend/.ruff_cache/	Analyse de qualité	Ruff

Annexe B - Checklist de maîtrise

- ☐ Je sais dessiner le trajet navigateur -> Nginx -> FastAPI -> PostgreSQL.
- ☐ Je sais expliquer l'inscription, bcrypt, le JWT et le Bearer token.
- ☐ Je sais distinguer modèle SQLAlchemy, schéma Pydantic et type TypeScript.
- ☐ Je sais expliquer route, service et repository.
- ☐ Je sais décrire ce que fait réellement chaque scanner.
- ☐ Je sais expliquer pourquoi un résultat safe n'est pas une garantie.
- ☐ Je sais démarrer, tester, lire les logs et arrêter Docker.
- ☐ Je sais ce qui est persistant et ce qui est généré.
- ☐ Je sais énumérer les limites avant adaptation OT.
- ☐ Je sais présenter la prochaine phase sans promettre du code prématuré.